iArsenic training data: from the Node loader to Python, and adding ARRP

This notebook walks through the work step by step and measures the difference it makes to model6.

1. The new Python loader gives exactly the same training data as the old Node loader (`preprocessing/lib/load-data.js`).
2. Why the ARRP data needed fixing: its depths are in feet and its region names don't match the map.
3. Locating ARRP wells by GPS instead of by name.
4. The name correction tool.
5. A faster copy of model6's prediction step, checked to give identical output.
6. Before and after: model6 trained on each version of the data, scored on held-out wells and on the independent BGD Traverse and VGQD test sets.
7. A quick look at what more the data could give: gradient boosting on nearest-neighbour features, with and without model6's expert patch.

Every number here is computed in this notebook from the files in the repo, except the model runs (`run_model.py` , `potential.py`), which are read from `runs/` .

In [1]:
```
import hashlib
import subprocess
import sys
import tempfile
from pathlib import Path

import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

HERE = Path.cwd()
REPO = HERE.parents[1]
IARSENIC = Path('/home/kane/Default Folder/code/iArsenic2024')
MODEL6 = IARSENIC / 'preprocessing/models/model6'
sys.path.insert(0, str(REPO))

pd.set_option('display.max_colwidth', 80)
pd.set_option('display.width', 200)

# chart styling: recessive axes and grid, neutral text, one colour per model
```

```
plt.rcParams.update({
    'figure.facecolor': '#fcfcfb', 'axes.facecolor': '#fcfcfb',
    'axes.edgecolor': '#c3c2b7', 'axes.labelcolor': '#52514e', 'text.color': '#0b0b0b',
    'xtick.color': '#52514e', 'ytick.color': '#52514e',
    'axes.grid': True, 'grid.color': '#e8e7e2', 'grid.linewidth': 0.8, 'axes.axisbelow': True,
    'axes.spines.top': False, 'axes.spines.right': False, 'font.size': 10,
})
MODEL_COLOURS = {'A': '#2a78d6', 'B': '#eb6834', 'C': '#1baf7a', 'C1': '#eda100', 'C2': '#e87ba4'}
MODEL_NAMES = {
    'A': 'A: pre-ARRP',
    'B': 'B: production (ARRP names uncorrected, feet)',
    'C': 'C: new (ARRP located by GPS, metres)',
    'C1': 'C1: C, one test per ARRP well',
    'C2': 'C2: C1, ARRP balanced against older data',
}


def sha256(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for block in iter(lambda: f.read(1 << 20), b''):
            h.update(block)
    return h.hexdigest()
```

1. The Python loader matches the Node loader

The old pipeline built model6's `training-data.json` with `node preprocessing/cli/csv-to-json.js data/*.csv`. Below, both the Node loader and the new `prep.py build` are run on the same four source files, and the outputs are compared byte for byte.

In [2]:
```
tmp = Path(tempfile.mkdtemp())
sources = ['1Mdataset', 'BGS_Ravenscroft', 'Danida-20220514', 'mouza-names']

subprocess.run(
    ['node', 'preprocessing/cli/csv-to-json.js', '-p', *[f'data/{s}.csv' for s in sources], '-o', str(tmp / 'js.json')],
    cwd=IARSENIC, check=True, capture_output=True,
)
subprocess.run(
    [sys.executable, 'prep.py', 'build', '-s', *sources, '-f', 'json', 'csv', '-o', str(tmp / 'py')],
    cwd=REPO, check=True, capture_output=True,
)

comparison = pd.DataFrame([
    ('Node csv-to-json.js (run now)', sha256(tmp / 'js.json')),
    ('Python prep.py build (run now)', sha256(tmp / 'py' / 'training-data.json')),
    ('model6 output/training-data.pre-arrp.json (historic)', sha256(MODEL6 / 'output/training-data.pre-arrp.json')),
], columns=['file', 'sha256'])
comparison['identical'] = comparison['sha256'] == comparison['sha256'][0]
comparison
```

Out[2]:

	file	sha256	identical
0	Node csv-to-json.js (run now)	c69e4625c3c6bb435f615024e6101dc7d66b79af79f610741cc789dc369ca994	True
1	Python prep.py build (run now)	c69e4625c3c6bb435f615024e6101dc7d66b79af79f610741cc789dc369ca994	True
2	model6 output/training-data.pre-arrp.json (historic)	c69e4625c3c6bb435f615024e6101dc7d66b79af79f610741cc789dc369ca994	True

The name-correction step was checked the same way. The old interactive tool (`find-name-corrections.js`) and the Python port were driven with the same 568 answers on the Danida data. Every one of the 569 screens was identical, and so was the corrections file they wrote. With ARRP and the feet conversion turned off, the Python build is byte-identical to the current production `training-data.json` too.

2. What was wrong with ARRP

ARRP is the DPHE screening export: 6.5 million wells, eight times the rest of the data put together. It was added to production by renaming its columns (`iArsenicArrpTools/preprocess_arrp.py`), which left two problems.

Depths are in feet

The other sources are in metres. ARRP's depths only make sense in feet: deep tube wells have a median of 750 (229 m, typical for a deep tube well; 750 m would be implausible) and ring wells 30 (9 m).

In [3]:
```
arrp = pd.read_csv(REPO / 'data/sources/ARRP_arsenic_master_all_data.csv', usecols=['Depth', 'WaterPointType'], low_memory=False)
arrp['Depth'] = pd.to_numeric(arrp['Depth'], errors='coerce')
depths = arrp.groupby('WaterPointType')['Depth'].agg(wells='count', median='median', p90=lambda d: d.quantile(0.9))
depths = depths.sort_values('wells', ascending=False).head(8)
depths['median in metres if feet'] = (depths['median'] * 0.3048).round(0)
depths
```

Out[3]:

WaterPointType	wells	median	p90	median in metres if feet
Shallow Tube Well No. 6 pump	5118014	90.0	230.0	27.0
Deep Tube Well No. 6 pump	731777	750.0	950.0	229.0
Shallow Tube Well Other Pumps	419653	140.0	280.0	43.0
Shallow Tube Well Tara dev head pump/Tara Pump	190427	130.0	240.0	40.0
Deep Tube Well Other Pumps	57685	690.0	870.0	210.0
Deep Tube Well Tara dev head pump/Tara pump	20143	700.0	900.0	213.0
Pipe Water System	13021	120.0	220.0	37.0
Ring Well No.6 pump	7896	30.0	45.0	9.0

Region names don't match the map

model6 joins wells to the mouza map by the full name `Division-District-Upazila-Union-Mouza`, exactly. ARRP's names are in capitals (`JESSORE`, `BEMARTA`), use different spellings, and include upazilas created after the map was made (Lalmai). The table shows the share of each training data set's wells whose name matches a mouza on model6's map.

In [4]:
```
mouza_map = gpd.read_file(REPO / 'data/geodata/mouzas.geojson', ignore_geometry=True)
levels = ['Division', 'District', 'Upazila', 'Union', 'Mouza']
map_paths = [set(map(tuple, mouza_map[['div', 'dis', 'upa', 'uni', 'mou']].iloc[:, :i + 1].to_numpy())) for i in range(5)]


def read_train(name):
    return pd.read_csv(HERE / f'data/{name}-train.csv', dtype={c: str for c in levels}, keep_default_na=False)


rows = []
for name in ['A', 'B', 'C']:
    wells = read_train(name)
    names = wells[levels].to_numpy()
    row = {'training data': MODEL_NAMES[name], 'wells': len(wells)}
    for i, level in enumerate(levels):
        row[f'matches map to {level.lower()}'] = np.mean([tuple(n[:i + 1]) in map_paths[i] for n in names])
    rows.append(row)
    del wells, names

matches = pd.DataFrame(rows)
matches['wells matching the map'] = (matches['wells'] * matches['matches map to mouza']).round().astype(int)
matches.style.format({c: '{:.1%}' for c in matches.columns if c.startswith('matches')} | {'wells': '{:,}', 'wells matching the map':
'{:,}'}).hide(axis='index')
```

Out[4]:

training data	wells	matches map to division	matches map to district	matches map to upazila	matches map to union	matches map to mouza
A: pre-ARRP	694,035	100.0%	100.0%	100.0%	98.6%	98.4%
B: production (ARRP names uncorrected, feet)	5,951,424	100.0%	36.4%	12.8%	11.6%	11.5%
C: new (ARRP located by GPS, metres)	5,942,473	100.0%	100.0%	100.0%	99.8%	99.8%

In production (B), adding ARRP took the training data from 0.87M to 7.4M wells, but **not one ARRP well matches the map down to mouza level**: the wells that match are exactly A's. ARRP's union and mouza names are in capitals, and model6 only uses a well whose full name matches a map mouza exactly, so it never sees ARRP.

3. Locating ARRP wells by GPS

Every ARRP well has coordinates. Rather than correcting tens of thousands of names by hand, the name-correction tool can mark a region `~gps~`. At build time its wells are placed on model6's own mouza map:

- division, district, upazila and union come from the mouza polygon the well is in;
- the mouza is the well's own mouza name if that union on the map has it (ignoring case), otherwise the polygon the well is in, because coordinates and mouza borders are both approximate.

ARRP's corrections file (`data/name-corrections/ARRP_arsenic_master_all_data-corrections.csv`) marks all eight divisions `~gps~`, so all of ARRP is located this way. A source's own corrections file only applies to that source, so the other data sets are unchanged.

Lalmai shows why names alone can't do this. It is an upazila created from parts of Comilla Sadar Dakshin and Laksam after the map was drawn:

In [5]:
```
from iarsenic_data.gps import locate, map_names

columns = ['UpazilaName', 'UnionName', 'WordName', 'Latitude', 'Longitude']
chunks = pd.read_csv(REPO / 'data/sources/ARRP_arsenic_master_all_data.csv', usecols=columns, dtype=str, chunksize=1_000_000)
lalmai = pd.concat(chunk[chunk['UpazilaName'].str.upper() == 'LALMAI'] for chunk in chunks)
lalmai[['Latitude', 'Longitude']] = lalmai[['Latitude', 'Longitude']].apply(pd.to_numeric, errors='coerce')
lalmai = lalmai[lalmai['Latitude'].between(20, 27) & lalmai['Longitude'].between(88, 93)]

rows = locate(lalmai['Latitude'].to_numpy(), lalmai['Longitude'].to_numpy())
names = map_names(rows[rows >= 0])
lalmai = lalmai[rows >= 0].assign(map_upazila=names[:, 2], map_union=names[:, 3])

where = lalmai.groupby('UnionName').apply(
    lambda g: ', '.join(f'{k} {v:.0%}' for k, v in (g.map_upazila + ' / ' + g.map_union).value_counts(normalize=True).head(2).items())
)
where.rename('where the wells are on the map').to_frame()
```

```
Loading mouza map mouzas.geojson
```

Out[5]:

UnionName	where the wells are on the map
BAGHMARA DAKSHIN	Comilla Sadar Dakshin / Baghmara 100%, Comilla Sadar Dakshin / Bara Para 0%
BAGHMARA UTTAR	Comilla Sadar Dakshin / Baghmara 80%, Comilla Sadar Dakshin / Baghmara Uttar...
BAKAI UTTAR	Laksam / Bakai 83%, Laksam / Laksam Paurashava 11%
BELGHAR DAKSHIN	Comilla Sadar Dakshin / Belghar 100%
BELGHAR UTTAR	Comilla Sadar Dakshin / Belghar 77%, Comilla Sadar Dakshin / Belghar Uttar 12%
BHOLAIN (UTTAR)	Comilla Sadar Dakshin / Bholain (Uttar) 96%, Comilla Sadar Dakshin / Bholain...
PERUL (DAKSHIN)	Comilla Sadar Dakshin / Perul (Dakshin) 92%, Laksam / Laksam Paurashava 4%
PERUL (UTTAR)	Comilla Sadar Dakshin / Perul (Uttar) 97%, Comilla Sadar Dakshin / Bholain (...

`BAKAI UTTAR`'s wells are under Laksam, not Comilla Sadar Dakshin. And `BAGHMARA UTTAR`'s are mostly in the map's `Baghmara`, not `Baghmara Uttar`, so correcting by name would put them in the wrong union. Below, Lalmai's ARRP wells are coloured by the map upazila they fall in (thick lines are upazila borders, thin lines union borders):

In [6]:

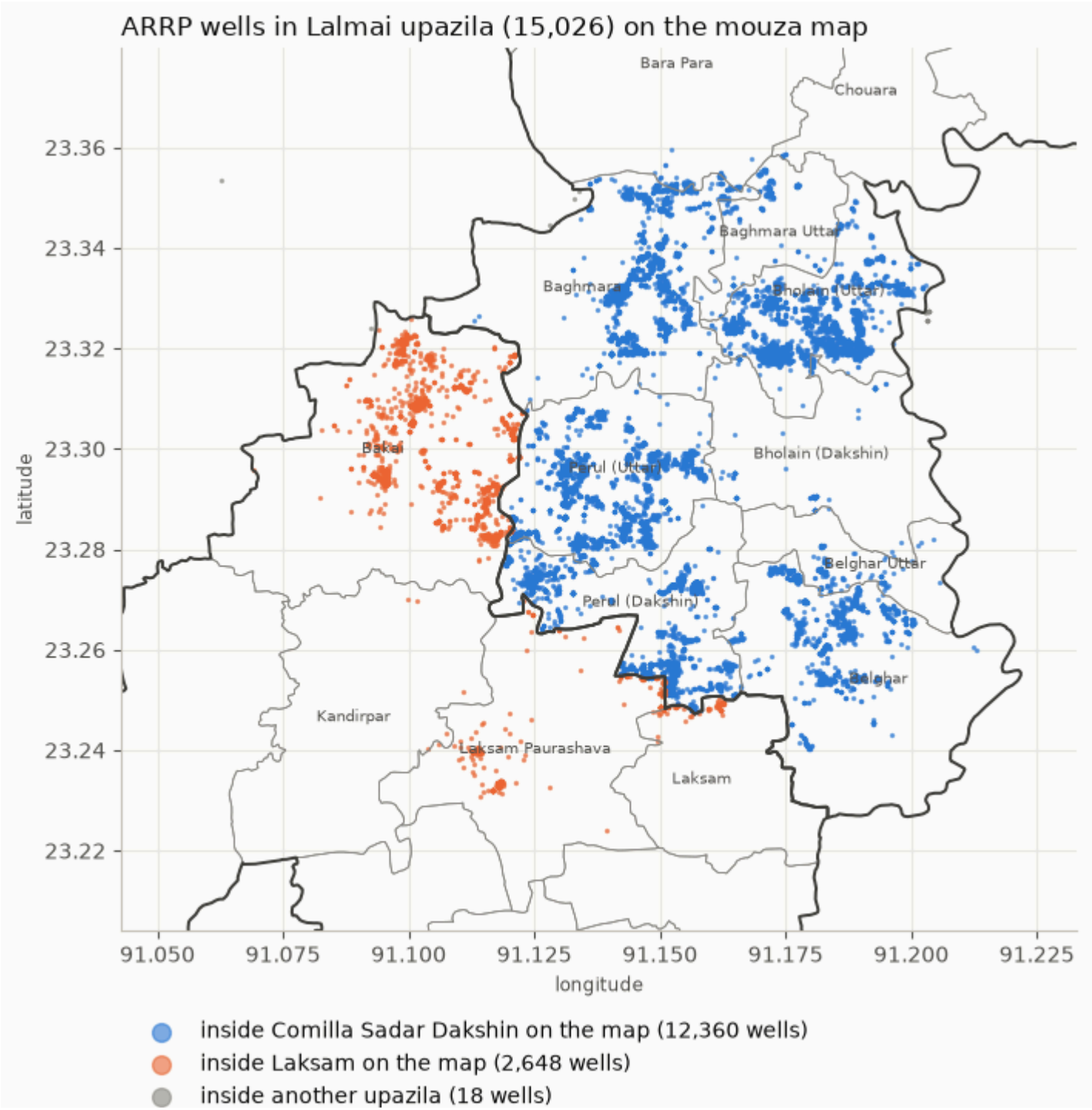
```
mouza_shapes = gpd.read_file(REPO / 'data/geodata/mouzas.geojson')
area = mouza_shapes[mouza_shapes['upa'].isin(['Comilla Sadar Dakshin', 'Laksam']) & (mouza_shapes['dis'] == 'Comilla')]
unions = area.dissolve(by=['upa', 'uni']).reset_index()

fig, ax = plt.subplots(figsize=(8, 8))
unions.boundary.plot(ax=ax, color='#8a8984', linewidth=0.6)
area.dissolve(by='upa').boundary.plot(ax=ax, color='#3d3c39', linewidth=1.4)

for upazila, colour in [('Comilla Sadar Dakshin', '#2a78d6'), ('Laksam', '#eb6834')]:
    wells = lalmai[lalmai['map_upazila'] == upazila]
    ax.scatter(wells['Longitude'], wells['Latitude'], s=2, color=colour, alpha=0.6,
               label=f'inside {upazila} on the map ({len(wells):,} wells)')
others = lalmai[~lalmai['map_upazila'].isin(['Comilla Sadar Dakshin', 'Laksam'])]
ax.scatter(others['Longitude'], others['Latitude'], s=2, color='#8a8984', alpha=0.6,
           label=f'inside another upazila ({len(others):,} wells)')

for _, u in unions.iterrows():
    if u.geometry.intersects(gpd.GeoSeries(gpd.points_from_xy(lalmai.Longitude, lalmai.Latitude)).union_all().convex_hull):
        p = u.geometry.representative_point()
        ax.annotate(u['uni'], (p.x, p.y), fontsize=7, color='#52514e', ha='center')

minx, miny, maxx, maxy = lalmai['Longitude'].min(), lalmai['Latitude'].min(), lalmai['Longitude'].max(), lalmai['Latitude'].max()
ax.set_xlim(minx - 0.02, maxx + 0.02)
ax.set_ylim(miny - 0.02, maxy + 0.02)
ax.set_title(f'ARRP wells in Lalmai upazila ({len(lalmai):,}) on the mouza map', loc='left')
ax.set_xlabel('longitude')
ax.set_ylabel('latitude')
ax.legend(loc='upper left', bbox_to_anchor=(0, -0.08), markerscale=6, frameon=False)
ax.set_aspect('equal')
plt.show()
```


ARRP wells in Lalmai upazila (15,026) on the mouza map
Bara Para
Chouara
Baghmara Uttar
Baghmara
Bholain (Uttar)
Bakai
Perul (Uttar)
Bholain (Dakshin)
Belghar Uttar
Perul (Dakshin)
Belghar
Kandirpar
Laksam Paurashava
Laksam
23.36
23.34
23.32
23.30
23.28
23.26
23.24
23.22
latitude
91.050
91.075
91.100
91.125
91.150
91.175
91.200
91.225
longitude
inside Comilla Sadar Dakshin on the map (12,360 wells)
inside Laksam on the map (2,648 wells)
inside another upazila (18 wells)

With the GPS corrections, 99.8% of the new training data (C, table above) matches a map mouza.

How precise is ARRP's GPS?

ARRP doesn't record GPS accuracy, so these are estimates from a sample of 400,000 ARRP wells. The coordinates are not office or mouza-centre placeholders: almost all wells are on their own point. When a well's GPS point is not inside the mouza it is named after, it is usually within about a mouza's width of it, which is the scale of the simplified map's borders. So GPS is trusted for the union, and the well's own mouza name is used where it fits.

In [7]:

```
sample = pd.read_csv(REPO / 'data/sources/ARRP_arsenic_master_all_data.csv',
                     usecols=['WordName', 'Latitude', 'Longitude'], dtype=str).sample(400_000, random_state=1)
sample[['Latitude', 'Longitude']] = sample[['Latitude', 'Longitude']].apply(pd.to_numeric, errors='coerce')
sample = sample[sample['Latitude'].between(20, 27) & sample['Longitude'].between(88, 93)]

points = gpd.GeoSeries(gpd.points_from_xy(sample['Longitude'], sample['Latitude']), crs=4326).to_crs(32645)
shared = sample.groupby(['Latitude', 'Longitude'])['WordName'].transform('size')

projected = mouza_shapes.to_crs(32645)
key = projected['div'] + '#' + projected['dis'] + '#' + projected['upa'] + '#' + projected['uni']
geometry_by_name = dict(zip(key + '#' + projected['mou'].str.casefold(), projected.geometry))

rows = locate(sample['Latitude'].to_numpy(), sample['Longitude'].to_numpy())
inside = rows >= 0
in_mouza = projected.iloc[rows[inside]]
names = sample['WordName'].str.casefold().to_numpy()[inside]
named = key.to_numpy()[rows[inside]] + '#' + names

named_exists = np.array([n in geometry_by_name for n in named])
inside_named = named_exists & (in_mouza['mou'].str.casefold().to_numpy() == names)
outside = named_exists & ~inside_named
distance = np.array([geometry_by_name[n].distance(p) for n, p in zip(named[outside], points.to_numpy()[inside][outside])])
area = projected.geometry.area
area = area[area > 5e4]

pd.DataFrame([
    ('wells on their own GPS point', f'{(shared == 1).mean():.1%}'),
    ('wells sharing a point with 10 or more others', f'{(shared > 10).mean():.2%}'),
    ('wells whose mouza name is a mouza in the map union they are in', f'{named_exists.mean():.1%}'),
    ('of those, inside that mouza', f'{inside_named.sum() / named_exists.sum():.1%}'),
    ('distance from the named mouza when outside it: median', f'{np.median(distance):.0f} m'),
    ('distance from the named mouza when outside it: 90%', f'{np.quantile(distance, 0.9):.0f} m'),
    ('typical mouza on the map (median area, ignoring slivers from the map fixes)', f'{area.median() / 1e6:.2f} km²'),
], columns=['', 'ARRP sample']).style.hide(axis='index')
```

Out[7]:

	ARRP sample
wells on their own GPS point	90.8%
wells sharing a point with 10 or more others	0.10%
wells whose mouza name is a mouza in the map union they are in	59.5%
of those, inside that mouza	57.7%
distance from the named mouza when outside it: median	396 m
distance from the named mouza when outside it: 90%	1984 m
typical mouza on the map (median area, ignoring slivers from the map fixes)	1.24 km²

4. The name correction tool

`python prep.py corrections <file>` is the port of `find-name-corrections.js`, with additions:

- a progress bar;
- `mouza-names.csv` as the reference of correct names by default;
- names that only differ in case from exactly one correct name are fixed automatically, unless GPS puts the wells somewhere else;
- each option shows how many of the region's subregions it shares and what share of its wells GPS puts there, with the best one suggested;
- option `g` records `~gps~` for regions whose wells have coordinates.

A screen from the ARRP data:

```
[------------------------------] 64 / 50,406 regions checked (0.1%)
...
6 Kotwali 14/15 shared  <- suggested GPS 99% (Arabpur, Basundia, Chanchra, Churamankati, Diara, Fatehpur, ...)
...
g Identify by GPS (37,508 of 37,508 records have coordinates, on the map they are in: Jessore -> Kotwali 99%, ...)

In Khulna -> Jessore
Incorrect upazila: JESSORE SADAR (ARABPUR, BASUNDIA, CHANCHRA, CHURAMANKATI, DIARA, FATHEHPUR, ...)
Input ID of correct spelling:
```

With `--exact-case` it behaves exactly like the old JS tool.

5. A faster model6, with identical output

model6's `generate_prediction_data.py` filters the whole training table for every mouza and depth band. With 7.4M wells that takes days: the earlier ARRP run took three weeks. `fast_model6.py` runs the same pandas operations on each region's rows only, looked up

from an index, and filters by depth before copying rows. The original model6 code was not changed.

`check_fast_model6.py` ran the original and the fast copy on the same training data for a sample of upazilas (single-mouza city upazilas, sparse hill tracts that widen to 100 km, and Comilla Sadar Dakshin with 525 mouzas), and compared the files byte for byte. The final version of the fast copy was checked on all of them:

In [8]:
```
for log in ['check-C.log', 'check-C-v2.log', 'check-A-v2.log']:
    print(f'--- {log}')
    print((HERE / 'runs' / log).read_text())
```

```
--- check-C.log
Original model6 code on 6 upazilas
  Chittagong-Chittagong-Pahartali.json: 1 mouzas in 6s
  Dhaka-Dhaka-Dakshinkhan.json: 1 mouzas in 8s
  Chittagong-Bandarban-Thanchi.json: 24 mouzas in 128s
  Chittagong-Bandarban-Ruma.json: 36 mouzas in 260s
  Khulna-Bagerhat-Mongla.json: 89 mouzas in 804s
  Chittagong-Comilla-Comilla Sadar Dakshin.json: 525 mouzas in 2348s
Fast copy
Generating 6 upazila files into /tmp/claude-1000/-home-kane/f195b7e4-f2f8-4cf8-9ba8-74eefb9628a8/scratchpad/check-C/fast
  300s
IDENTICAL: Chittagong-Bandarban-Ruma.json
IDENTICAL: Chittagong-Bandarban-Thanchi.json
IDENTICAL: Chittagong-Chittagong-Pahartali.json
IDENTICAL: Chittagong-Comilla-Comilla Sadar Dakshin.json
IDENTICAL: Dhaka-Dhaka-Dakshinkhan.json
IDENTICAL: Khulna-Bagerhat-Mongla.json
All identical

--- check-C-v2.log
Original model6 code on 4 upazilas
  Chittagong-Chittagong-Pahartali.json: 1 mouzas in 6s
  Dhaka-Dhaka-Dakshinkhan.json: 1 mouzas in 7s
  Chittagong-Bandarban-Thanchi.json: 24 mouzas in 102s
  Chittagong-Bandarban-Ruma.json: 36 mouzas in 144s
Fast copy
Generating 4 upazila files into /tmp/claude-1000/-home-kane/f195b7e4-f2f8-4cf8-9ba8-74eefb9628a8/scratchpad/check-C2/fast
  13s
IDENTICAL: Chittagong-Bandarban-Ruma.json
IDENTICAL: Chittagong-Bandarban-Thanchi.json
IDENTICAL: Chittagong-Chittagong-Pahartali.json
IDENTICAL: Dhaka-Dhaka-Dakshinkhan.json
All identical
Mongla, Comilla Sadar Dakshin (C data): fixed fast copy vs original model6 code
IDENTICAL: Khulna-Bagerhat-Mongla.json
IDENTICAL: Chittagong-Comilla-Comilla Sadar Dakshin.json

--- check-A-v2.log
Original model6 code on 3 upazilas
  Chittagong-Bandarban-Thanchi.json: 24 mouzas in 11s
  Khulna-Bagerhat-Mongla.json: 89 mouzas in 49s
  Sylhet-Sylhet-Zakiganj.json: 259 mouzas in 128s
Fast copy
Generating 3 upazila files into /tmp/claude-1000/-home-kane/f195b7e4-f2f8-4cf8-9ba8-74eefb9628a8/scratchpad/check-A2/fast
  11s
IDENTICAL: Chittagong-Bandarban-Thanchi.json
IDENTICAL: Khulna-Bagerhat-Mongla.json
IDENTICAL: Sylhet-Sylhet-Zakiganj.json
All identical
```

6. Before and after

Three training data sets, built by `datasets.py`:

	training data
A	pre-ARRP: 1Mdataset, BGS/Ravenscroft, Danida, mouza-names (what the Node loader used)
B	production: A + ARRP as it is in production now (names uncorrected, depths in feet)
C	new: A + ARRP located by GPS, depths in metres

The same 20% of wells are held out of all three: a hash of (source, row) picks them, except for ARRP, where 19% of records are repeat tests of a well, so its wells are split by `TubeWellCode` and all the tests of a well are on the same side. model6 was run on each training set with `run_model.py`.

Since none of B's ARRP wells match the map (section 2), model6 trained on B is the same model as on A, file for file:

In [9]:
```
import filecmp

a_files = sorted(p.name for p in (HERE / 'runs/A').glob('*.json'))
different = [f for f in a_files if not filecmp.cmp(HERE / 'runs/A' / f, HERE / 'runs/B' / f, shallow=False)]
print(f'{len(a_files)} upazila files, {len(different)} different between A and B')
```

```
538 upazila files, 0 different between A and B
```

Is ARRP biased?

Before scoring, a look at whether ARRP over-represents some wells. First, repeat tests and duplicates:

In [10]:
```
arrp_tests = pd.read_csv(REPO / 'data/sources/ARRP_arsenic_master_all_data.csv',
                         usecols=['TubeWellCode', 'TestingDate', 'TestResult', 'TestStatus', 'Depth', 'Latitude', 'Longitude'],
                         dtype=str, keep_default_na=False)
arrp_tests['arsenic'] = pd.to_numeric(arrp_tests['TestResult'], errors='coerce')
tests_per_well = arrp_tests['TubeWellCode'].map(arrp_tests['TubeWellCode'].value_counts())
by_well = arrp_tests.groupby('TubeWellCode')['arsenic']
repeat = by_well.size() > 1
low, high = by_well.min()[repeat], by_well.max()[repeat]
exact = arrp_tests.duplicated(['TubeWellCode', 'TestingDate', 'TestResult', 'Depth', 'Latitude', 'Longitude'])

pd.DataFrame([
    ('records', f'{len(arrp_tests):,}'),
    ('distinct wells (TubeWellCode)', f'{arrp_tests.TubeWellCode.nunique():,}'),
    ('exact duplicate records', f'{exact.sum():,} ({exact.mean():.1%})'),
    ('wells tested more than once', f'{repeat.sum():,}'),
    ('records that are repeat tests', f'{len(arrp_tests) - arrp_tests.TubeWellCode.nunique():,}'),
    ('repeat-tested wells whose tests disagree about > 10 ppb', f'{((low <= 10) & (high > 10)).mean():.1%}'),
    ('wells over 10 ppb in any test: repeat-tested wells', f'{(high > 10).mean():.1%}'),
    ('wells over 10 ppb in any test: all wells', f'{(by_well.max() > 10).mean():.1%}'),
    ('records over 10 ppb: Approved / Pending', ' / '.join(f'{(arrp_tests.arsenic[arrp_tests.TestStatus == s] > 10).mean():.1%}' for s in ['Approved',
```

```
'Pending'])),
], columns=['', 'ARRP']).style.hide(axis='index')
```

Out[10]:

	ARRP
records	6,572,636
distinct wells (TubeWellCode)	5,835,274
exact duplicate records	16,047 (0.2%)
wells tested more than once	538,522
records that are repeat tests	737,362
repeat-tested wells whose tests disagree about > 10 ppb	27.1%
wells over 10 ppb in any test: repeat-tested wells	49.7%
wells over 10 ppb in any test: all wells	36.0%
records over 10 ppb: Approved / Pending	31.7% / 42.2%

Exact duplicates are rare, but 11% of ARRP's records are repeat tests of a well. Repeat tests are noisy (a quarter disagree about 10 ppb), and retested wells are more polluted than the rest, so repeats push ARRP towards *unsafe*, not safe. Results still `Pending` approval are more polluted than `Approved` ones, so keeping only approved results would make ARRP look safer still.

Then, in the same mouza and model6 depth band, how ARRP compares with the older data:

In [11]:

```
from iarsenic_data.load_data import load_records

located = load_records()
located = located[located['is_well']]
band = np.digitize(located['Depth'], [15.3, 45, 65, 90, 150])
located = located.assign(cell=located[levels].agg('#'.join, axis=1) + '#' + band.astype(str))
short = {'ARRP_arsenic_master_all_data': 'ARRP', '1Mdataset': '1M', 'BGS_Ravenscroft': 'BGS', 'Danida-20220514': 'Danida'}
located['source'] = located['Source'].map(short)

cells = located.groupby(['cell', 'source'])['Arsenic'].agg(
    n='size', gt10=lambda a: (a > 10).mean(), gt50=lambda a: (a > 50).mean()).unstack('source')
rows = []
for other in ['1M', 'Danida', 'BGS']:
    both = cells[cells['n']['ARRP'].notna() & cells['n'][other].notna()]
    weight = both['n'][other]
    rows.append({
        'compared with': other, 'places with both': len(both),
        'ARRP wells per other well': both['n']['ARRP'].sum() / weight.sum(),
        f'> 10 ppb: other': np.average(both['gt10'][other], weights=weight),
        f'> 10 ppb: ARRP': np.average(both['gt10']['ARRP'], weights=weight),
        f'> 50 ppb: other': np.average(both['gt50'][other], weights=weight),
        f'> 50 ppb: ARRP': np.average(both['gt50']['ARRP'], weights=weight),
    })
comparison = pd.DataFrame(rows)
comparison.style.format({'places with both': '{:,}', 'ARRP wells per other well': '{:.1f}x'} |
                        {c: '{:.1%}' for c in comparison.columns if 'ppb' in c}).hide(axis='index')
```

```
Loading 1Mdataset.csv
  955892 records
Loading BGS_Ravenscroft.csv
  27608 records
Loading Danida-20220514.csv
  161064 records
Loading mouza-names.csv
  57093 records
Loading ARRP_arsenic_master_all_data.csv
  6572636 records
Using ARRP_arsenic_master_all_data-corrections.csv for ARRP_arsenic_master_all_data
GPS located 6561250 records (3894355 by their mouza name in the union they are in), dropped 2833 without usable coordinates and 8553 outside the map
Parsed 7774293 total records, 7487021 with location, 7429471 wells
```

Out[11]:

compared with	places with both	ARRP wells per other well	> 10 ppb: other	> 10 ppb: ARRP	> 50 ppb: other	> 50 ppb: ARRP
1M	14,414	1.3x	46.9%	38.1%	27.5%	17.8%
Danida	284	2.6x	83.6%	66.0%	67.2%	31.5%
BGS	3,993	30.4x	9.9%	11.3%	5.6%	3.5%

1M and Danida are field-kit surveys from about 20 years earlier; BGS is lab-measured, mostly deep wells. ARRP reads well below the older kit surveys in the same places. Where ARRP and older data overlap, ARRP usually has most of the wells, so it sets model6's medians there.

The two independent test sets are lab-measured, so they show how ARRP compares with lab results in the same unions and depth bands:

In [12]:

```
import evaluate

lab = pd.concat([evaluate.bgd_traverse_wells().assign(test_set='BGD Traverse'), evaluate.vgqd_wells().assign(test_set='VGQD')])
lab_band = np.digitize(lab['depth'], [15.3, 45, 65, 90, 150])
lab['place'] = lab['div'] + '#' + lab['dis'] + '#' + lab['upa'] + '#' + lab['uni'] + '#' + lab_band.astype(str)

located['place'] = located[levels[:4]].agg('#'.join, axis=1) + '#' + band.astype(str)
located['kind'] = np.where(located['source'] == 'ARRP', 'ARRP', 'older surveys')
nearby = located[located['place'].isin(set(lab['place']))]
kit = nearby.groupby(['place', 'kind'])['Arsenic'].agg(n='size', gt10=lambda a: (a > 10).mean()).unstack('kind')
kit.columns = [f'{a}: {b}' for a, b in kit.columns]

places = lab.groupby(['test_set', 'place'])['arsenic'].agg(n='size', gt10=lambda a: (a > 10).mean()).reset_index()
places = places.merge(kit, left_on='place', right_index=True, how='left')

rows = []
for test_set, group in places.groupby('test_set'):
    for kind in ['ARRP', 'older surveys']:
        has = group[f'n: {kind}'].notna()
        weight = group.loc[has, 'n']
        rows.append({
            'lab test set': test_set, 'compared with': kind,
            'places (union and depth band)': int(has.sum()),
```

```
            'wells compared with': int(group.loc[has, f'n: {kind}'].sum()),
            '> 10 ppb: lab': np.average(group.loc[has, 'gt10'], weights=weight),
            f'> 10 ppb: field kits': np.average(group.loc[has, f'gt10: {kind}'], weights=weight),
        })
lab_comparison = pd.DataFrame(rows)
lab_comparison.style.format({'wells compared with': '{:,}', '> 10 ppb: lab': '{:.0%}', '> 10 ppb: field kits': '{:.0%}'}).hide(axis='index')
```

```
/home/kane/Default Folder/code/iarsenic-training-data/experiments/model6-evaluation/evaluate.py:226: UserWarning: Geometry is in a geographic CRS. Results
from 'centroid' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this operation.

  centroids = gpd.GeoSeries(merged['geometry'], crs=unions.crs).to_crs('EPSG:4326').centroid
```

Out[12]:

lab test set	compared with	places (union and depth band)	wells compared with	> 10 ppb: lab	> 10 ppb: field kits
BGD Traverse	ARRP	76	68,284	52%	31%
BGD Traverse	older surveys	38	25,199	51%	51%
VGQD	ARRP	50	44,166	51%	32%
VGQD	older surveys	23	12,140	47%	49%

In the same places, the older field-kit surveys agree with the lab measurements, but **ARRP reads about 20 percentage points lower**. Either ARRP's field kits under-read (most of its results are 0, 5 or 10 ppb), or the lab-tested wells are not like the wells ARRP tested (older wells, or wells picked for study). Either way, ARRP's readings shouldn't be taken at face value without checking them against lab results.

Two more training sets test whether pruning ARRP helps anyway:

	training data
C1	C, keeping only each ARRP well's latest test
C2	C1, and in each mouza and depth band with older wells, at most as many ARRP wells as older wells (a random sample). Places with only ARRP keep all of it

So the comparison is A (what production effectively is) against C, C1 and C2.

Scoring. The main score is the dissertation's (`UoP-SoftEng-Dissertation/models/model_utils`), which reported model5 (model6 without the map fixes) at F1 0.82, accuracy 0.84 and specificity 0.85: a well is *polluted* above 10 ppb, and a prediction is polluted when the model's message is polluted or highly polluted (median above 10 ppb) or there isn't enough data. Wells are predicted as red-stained. Model6 A on the non-ARRP held-out wells reproduces those numbers. For comparison with `experiments/BGDEstimates`, the F1 at > 50 ppb (risk band 2 and up, the usual "unsafe") and > 100 ppb are given too.

Test sets:

- **holdout**: the held-out wells, split into the non-ARRP sources and ARRP, located as in C. They have no staining information, so they are predicted as red-stained wells, the prediction that uses the training data.

- **BGD Traverse** and **VGQD**: the independent lab-tested wells used in `experiments/BGDEstimates`, with their staining. Black-stained wells are always predicted safe whatever the training data, so the red-stained wells are also shown on their own. These are small (213 and 173 wells), so a few wells move the scores.

In [13]:
```
results = pd.read_csv(HERE / 'runs' / 'results.csv')
results = results[results['model'].isin(['A', 'C', 'C1', 'C2'])]
results['model'] = results['model'].map(MODEL_NAMES)

summary = results[['test set', 'model', 'wells', 'gt10_accuracy', 'gt10_precision', 'gt10_sensitivity',
                   'gt10_specificity', 'gt10_f1', 'gt50_f1', 'gt100_f1', 'risk_band_accuracy']].copy()
summary['model'] = summary['model'].map({MODEL_NAMES['A']: 'A: production', MODEL_NAMES['C']: 'C: with ARRP',
                                         MODEL_NAMES['C1']: 'C1: one test per well', MODEL_NAMES['C2']: 'C2: balanced'})
summary.columns = ['test set', 'model', 'wells', 'accuracy', 'precision', 'sensitivity', 'specificity', 'f1',
                   'f1 > 50 ppb', 'f1 > 100 ppb', 'risk band accuracy']
summary.style.format({c: '{:.3f}' for c in summary.columns if c not in ('test set', 'model', 'wells')} | {'wells': '{:,}'}).hide(axis='index')
```

Out[13]:

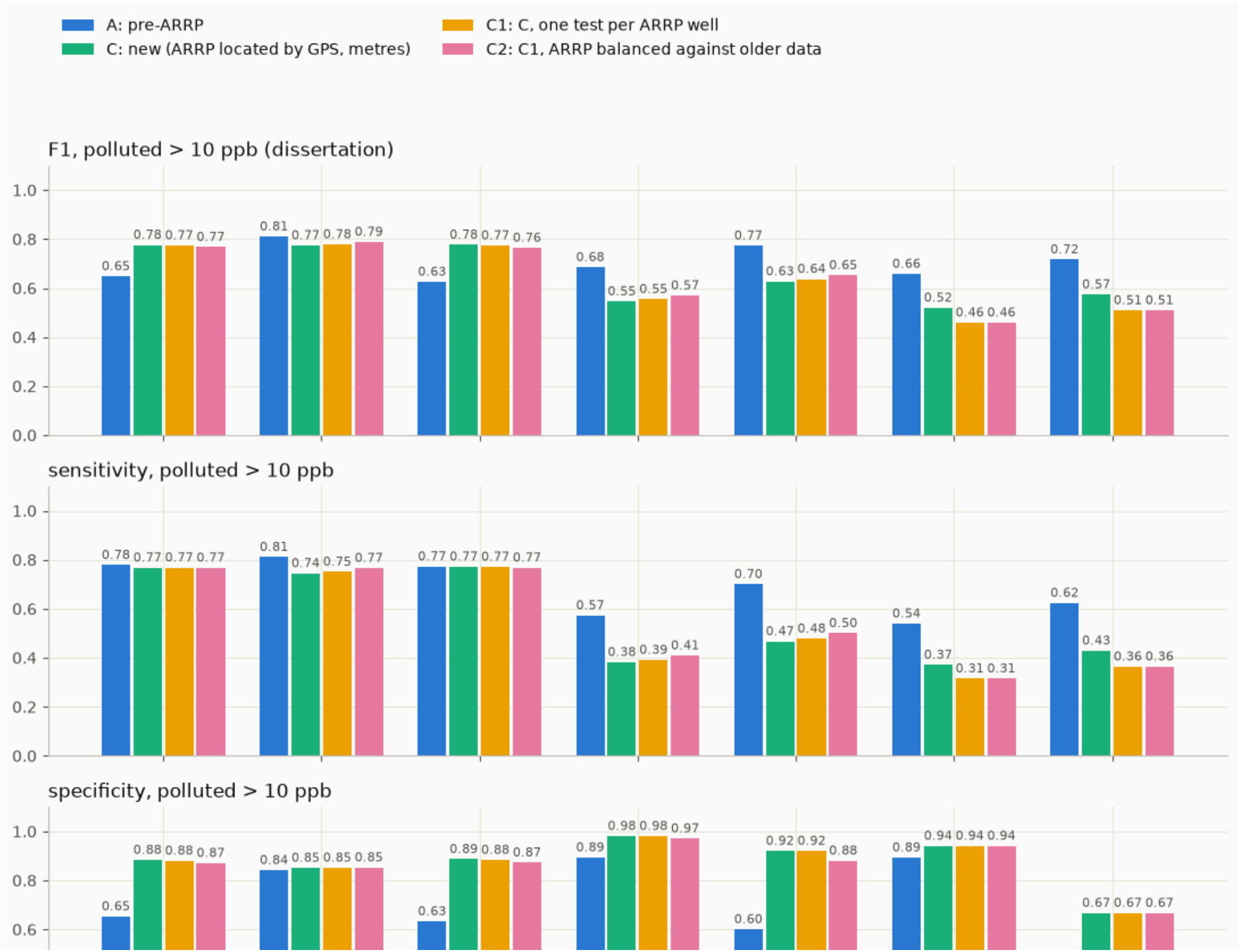
test set	model	wells	accuracy	precision	sensitivity	specificity	f1	f1 > 50 ppb	f1 > 100 ppb	risk band accuracy
holdout: all sources	A: production	1,486,998	0.698	0.555	0.778	0.653	0.648	0.473	0.377	0.294
holdout: non-ARRP sources	A: production	174,643	0.827	0.810	0.811	0.841	0.810	0.676	0.435	0.448
holdout: ARRP	A: production	1,312,355	0.681	0.525	0.772	0.632	0.625	0.445	0.370	0.274
BGD Traverse	A: production	213	0.728	0.851	0.573	0.893	0.685	0.647	0.556	0.455
BGD Traverse: red stained	A: production	115	0.678	0.863	0.700	0.600	0.773	0.714	0.606	0.217
VGQD	A: production	173	0.711	0.842	0.539	0.893	0.658	0.679	0.618	0.538
VGQD: red stained	A: production	92	0.587	0.842	0.623	0.400	0.716	0.718	0.667	0.304
holdout: all sources	C: with ARRP	1,486,998	0.842	0.785	0.768	0.883	0.776	0.647	0.533	0.485
holdout: non-ARRP sources	C: with ARRP	174,643	0.803	0.806	0.744	0.851	0.774	0.661	0.439	0.446
holdout: ARRP	C: with ARRP	1,312,355	0.847	0.782	0.772	0.886	0.776	0.645	0.549	0.490
BGD Traverse	C: with ARRP	213	0.671	0.955	0.382	0.981	0.545	0.596	0.448	0.516
BGD Traverse: red stained	C: with ARRP	115	0.565	0.955	0.467	0.920	0.627	0.674	0.500	0.322
VGQD	C: with ARRP	173	0.647	0.868	0.371	0.940	0.520	0.533	0.293	0.509
VGQD: red stained	C: with ARRP	92	0.467	0.868	0.429	0.667	0.574	0.571	0.324	0.239
holdout: all sources	C1: one test per well	1,486,998	0.840	0.781	0.768	0.880	0.774	0.646	0.532	0.486
holdout: non-ARRP sources	C1: one test per well	174,643	0.806	0.808	0.750	0.851	0.778	0.662	0.439	0.449
holdout: ARRP	C1: one test per well	1,312,355	0.844	0.776	0.771	0.883	0.774	0.643	0.547	0.491
BGD Traverse	C1: one test per well	213	0.676	0.956	0.391	0.981	0.555	0.590	0.421	0.512
BGD Traverse: red stained	C1: one test per well	115	0.574	0.956	0.478	0.920	0.637	0.667	0.471	0.313
VGQD	C1: one test per well	173	0.618	0.848	0.315	0.940	0.459	0.518	0.293	0.532
VGQD: red stained	C1: one test per well	92	0.413	0.848	0.364	0.667	0.509	0.557	0.324	0.283
holdout: all sources	C2: balanced	1,486,998	0.834	0.767	0.767	0.870	0.767	0.641	0.526	0.480
holdout: non-ARRP sources	C2: balanced	174,643	0.813	0.812	0.766	0.852	0.788	0.665	0.440	0.449
holdout: ARRP	C2: balanced	1,312,355	0.836	0.760	0.767	0.872	0.764	0.636	0.540	0.484
BGD Traverse	C2: balanced	213	0.681	0.938	0.409	0.971	0.570	0.593	0.414	0.493
BGD Traverse: red stained	C2: balanced	115	0.583	0.938	0.500	0.880	0.652	0.667	0.462	0.278
VGQD	C2: balanced	173	0.618	0.848	0.315	0.940	0.459	0.518	0.293	0.532
VGQD: red stained	C2: balanced	92	0.413	0.848	0.364	0.667	0.509	0.557	0.324	0.283

In [14]:

```
test_sets = list(dict.fromkeys(results['test set']))
metrics = [('gt10_f1', 'F1, polluted > 10 ppb (dissertation)'), ('gt10_sensitivity', 'sensitivity, polluted > 10 ppb'),
           ('gt10_specificity', 'specificity, polluted > 10 ppb'), ('gt50_f1', 'F1, unsafe > 50 ppb')]
shown = {k: v for k, v in MODEL_NAMES.items() if k in ('A', 'C', 'C1', 'C2')}

fig, axes = plt.subplots(len(metrics), 1, figsize=(11, 3.2 * len(metrics)), sharex=True)
x = np.arange(len(test_sets))
width = 0.2
for ax, (metric, title) in zip(axes, metrics):
    for i, (key, name) in enumerate(shown.items()):
        values = [results.loc[(results['test set'] == t) & (results['model'] == name), metric].iloc[0] for t in test_sets]
        bars = ax.bar(x + (i - (len(shown) - 1) / 2) * width, values, width - 0.02, color=MODEL_COLOURS[key], label=name)
        ax.bar_label(bars, fmt='%.2f', fontsize=8, color='#52514e', padding=2)
```

```
    ax.set_title(title, loc='left')
    ax.set_ylim(0, 1.1)
axes[-1].set_xticks(x, [t.replace(': ', ':\n') for t in test_sets])
axes[0].legend(loc='upper left', bbox_to_anchor=(0, 1.6), ncol=2, frameon=False)
plt.tight_layout()
plt.show()
```


A: pre-ARRP
C: new (ARRP located by GPS, metres)
C1: C, one test per ARRP well
C2: C1, ARRP balanced against older data
F1, polluted > 10 ppb (dissertation)
1.0
0.8
0.6
0.4
0.2
0.0
0.65 0.78 0.77 0.77
0.81 0.77 0.78 0.79
0.63 0.78 0.77 0.76
0.68 0.55 0.55 0.57
0.77 0.63 0.64 0.65
0.66 0.52 0.46 0.46
0.72 0.57 0.51 0.51
sensitivity, polluted > 10 ppb
1.0
0.8
0.6
0.4
0.2
0.0
0.78 0.77 0.77 0.77
0.81 0.74 0.75 0.77
0.77 0.77 0.77 0.77
0.57 0.38 0.39 0.41
0.70 0.47 0.48 0.50
0.54 0.37 0.31 0.31
0.62 0.43 0.36 0.36
specificity, polluted > 10 ppb
1.0
0.8
0.6
0.65 0.88 0.88 0.87
0.84 0.85 0.85 0.85
0.63 0.89 0.88 0.87
0.89 0.98 0.98 0.97
0.60 0.92 0.92 0.88
0.89 0.94 0.94 0.94
0.67 0.67 0.67


0.4
0.2
0.0
0.40
F1, unsafe > 50 ppb
1.0
0.8
0.6
0.4
0.2
0.0
0.47
0.65 0.65 0.64
0.68 0.66 0.66 0.67
0.45
0.64 0.64 0.64
0.65 0.60 0.59 0.59
0.71 0.67 0.67 0.67
0.68 0.53 0.52 0.52
0.72 0.57 0.56 0.56
holdout: all sources
holdout: non-ARRP sources
holdout: ARRP
BGD Traverse
BGD Traverse: red stained
VGQD
VGQD: red stained

7. A quick look at what more the data could give

`potential.py` asks how much better than model6 the same wells could be predicted, as a rough guide to what to pursue next (for example a CNN that looks at the wells around each place at several scales). It trains gradient boosting (LightGBM) on features computed from the training wells only, on a sample of the same held-out wells:

- **knn**: the arsenic of the 4, 16, 64 and 256 nearest training wells in the same model6 depth band, located by GPS where the data has it (ARRP, BGS) and at the mouza's centre otherwise;
- **patch**: model6's expert patch, the upazila's risk for wells <= 90 m and > 90 m;
- **env**: elevation, flood index, distance to river and geology from the 3D CNN/PointNet project's rasters, and the coordinates.

All tests of a well are left out of its neighbours. Each model's decision threshold is picked on part of its training sample. This is a quick look: one sample, default settings, and a random hold-out, which flatters methods that use nearby wells (a test well's neighbours are often from the same village). Holding out whole areas would show how much of the gain survives where data is sparse.

In [15]:
```
potential = pd.read_csv(HERE / 'runs' / 'potential-results.csv')
order = ['model6 A', 'model6 C', 'GBM: patch', 'GBM: env', '16 nearest wells in the depth band (no model)',
         'GBM: knn', 'GBM: knn + patch', 'GBM: knn + patch + env']
potential['order'] = potential['model'].map({m: i for i, m in enumerate(order)})
table = potential.sort_values(['target', 'test set', 'order'])[['target', 'test set', 'model', 'wells', 'accuracy',
                                                                'precision', 'sensitivity', 'specificity', 'f1', 'auc']]
table.style.format({c: '{:.3f}' for c in ['accuracy', 'precision', 'sensitivity', 'specificity', 'f1', 'auc']} | {'wells': '{:,}'},
na_rep='').hide(axis='index')
```

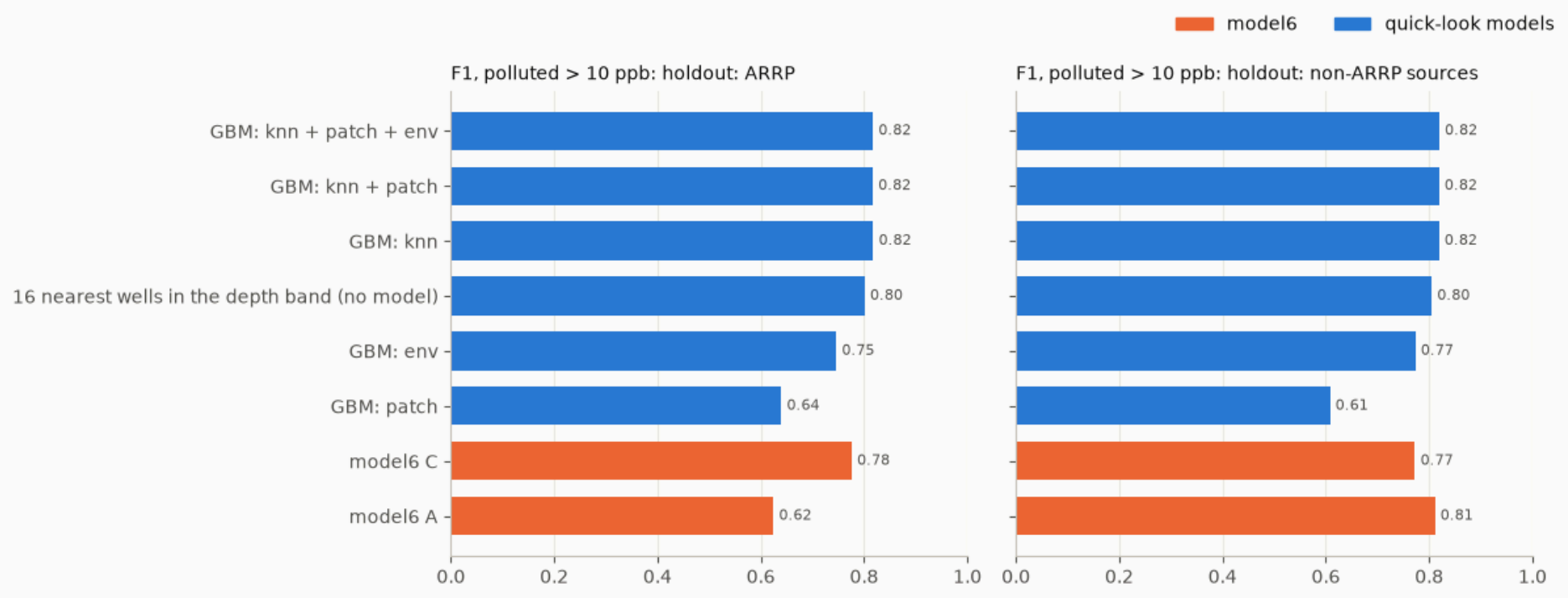
Out[15]:

target	test set	model	wells	accuracy	precision	sensitivity	specificity	f1	auc
polluted > 10 ppb	holdout: ARRP	model6 A	150,000	0.680	0.525	0.770	0.633	0.624	
polluted > 10 ppb	holdout: ARRP	model6 C	150,000	0.846	0.781	0.770	0.886	0.776	
polluted > 10 ppb	holdout: ARRP	GBM: patch	150,000	0.753	0.644	0.636	0.815	0.640	0.775
polluted > 10 ppb	holdout: ARRP	GBM: env	150,000	0.822	0.735	0.758	0.856	0.746	0.887
polluted > 10 ppb	holdout: ARRP	16 nearest wells in the depth band (no model)	150,000	0.865	0.815	0.787	0.906	0.801	0.905
polluted > 10 ppb	holdout: ARRP	GBM: knn	150,000	0.872	0.808	0.828	0.896	0.818	0.941
polluted > 10 ppb	holdout: ARRP	GBM: knn + patch	150,000	0.873	0.812	0.822	0.899	0.817	0.941
polluted > 10 ppb	holdout: ARRP	GBM: knn + patch + env	150,000	0.873	0.812	0.822	0.899	0.817	0.941
polluted > 10 ppb	holdout: non-ARRP sources	model6 A	59,939	0.828	0.810	0.812	0.842	0.811	
polluted > 10 ppb	holdout: non-ARRP sources	model6 C	59,939	0.801	0.805	0.743	0.850	0.773	
polluted > 10 ppb	holdout: non-ARRP sources	GBM: patch	60,000	0.713	0.800	0.490	0.898	0.608	0.760
polluted > 10 ppb	holdout: non-ARRP sources	GBM: env	60,000	0.798	0.789	0.758	0.832	0.773	0.880
polluted > 10 ppb	holdout: non-ARRP sources	16 nearest wells in the depth band (no model)	60,000	0.826	0.819	0.791	0.855	0.805	0.880
polluted > 10 ppb	holdout: non-ARRP sources	GBM: knn	60,000	0.833	0.806	0.833	0.833	0.819	0.919
polluted > 10 ppb	holdout: non-ARRP sources	GBM: knn + patch	60,000	0.834	0.809	0.830	0.837	0.819	0.919
polluted > 10 ppb	holdout: non-ARRP sources	GBM: knn + patch + env	60,000	0.834	0.812	0.828	0.840	0.820	0.919
unsafe > 50 ppb	holdout: ARRP	model6 A	150,000	0.648	0.302	0.850	0.607	0.446	
unsafe > 50 ppb	holdout: ARRP	model6 C	150,000	0.841	0.513	0.875	0.834	0.647	
unsafe > 50 ppb	holdout: ARRP	GBM: patch	150,000	0.823	0.479	0.727	0.842	0.578	0.840
unsafe > 50 ppb	holdout: ARRP	GBM: env	150,000	0.894	0.659	0.749	0.922	0.701	0.923
unsafe > 50 ppb	holdout: ARRP	16 nearest wells in the depth band (no model)	150,000	0.924	0.791	0.741	0.961	0.765	0.926
unsafe > 50 ppb	holdout: ARRP	GBM: knn	150,000	0.927	0.784	0.777	0.957	0.780	0.960
unsafe > 50 ppb	holdout: ARRP	GBM: knn + patch	150,000	0.927	0.784	0.777	0.957	0.780	0.960
unsafe > 50 ppb	holdout: ARRP	GBM: knn + patch + env	150,000	0.927	0.779	0.781	0.956	0.780	0.960
unsafe > 50 ppb	holdout: non-ARRP sources	model6 A	59,939	0.790	0.572	0.833	0.774	0.679	
unsafe > 50 ppb	holdout: non-ARRP sources	model6 C	59,939	0.786	0.571	0.790	0.785	0.663	
unsafe > 50 ppb	holdout: non-ARRP sources	GBM: patch	60,000	0.808	0.669	0.554	0.901	0.606	0.781
unsafe > 50 ppb	holdout: non-ARRP sources	GBM: env	60,000	0.835	0.703	0.659	0.899	0.681	0.877
unsafe > 50 ppb	holdout: non-ARRP sources	16 nearest wells in the depth band (no model)	60,000	0.846	0.737	0.655	0.915	0.693	0.868
unsafe > 50 ppb	holdout: non-ARRP sources	GBM: knn	60,000	0.850	0.722	0.709	0.901	0.716	0.906
unsafe > 50 ppb	holdout: non-ARRP sources	GBM: knn + patch	60,000	0.851	0.724	0.711	0.901	0.717	0.906
unsafe > 50 ppb	holdout: non-ARRP sources	GBM: knn + patch + env	60,000	0.850	0.715	0.724	0.895	0.720	0.907

In [16]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6), sharey=True)
target = 'polluted > 10 ppb'
models = [m for m in order if m in set(potential['model'])]
for ax, test_set in zip(axes, ['holdout: ARRP', 'holdout: non-ARRP sources']):
    rows = potential[(potential['target'] == target) & (potential['test set'] == test_set)].set_index('model').loc[models]
    colours = ['#eb6834' if m.startswith('model6') else '#2a78d6' for m in models]
    bars = ax.barh(range(len(models)), rows['f1'], color=colours, height=0.7)
```

```
    ax.bar_label(bars, fmt='%.2f', fontsize=8, color='#52514e', padding=3)
    ax.set_yticks(range(len(models)), models)
    ax.invert_yaxis()
    ax.set_xlim(0, 1)
    ax.set_title(f'F1, {target}: {test_set}', loc='left', fontsize=10)
    ax.grid(axis='y', visible=False)
fig.legend(handles=[plt.Rectangle((0, 0), 1, 1, color='#eb6834'), plt.Rectangle((0, 0), 1, 1, color='#2a78d6')],
           labels=['model6', 'quick-look models'], loc='upper right', frameon=False, ncol=2)
plt.tight_layout(rect=(0, 0, 1, 0.93))
plt.show()
```



Which features the full model relies on (share of the total gain):

In [17]:
```
importance = pd.read_csv(HERE / 'runs' / 'potential-importance.csv', index_col=0)['polluted > 10 ppb: knn + patch + env'].dropna()
importance = (importance / importance.sum()).sort_values(ascending=False)
groups = importance.groupby(lambda f: 'patch' if f.startswith('patch') else
                            'knn' if f.startswith(('band_', 'any_')) else
                            'depth' if f == 'depth' else 'env').sum().sort_values(ascending=False)
display(groups.rename('share of gain').to_frame().style.format('{:.1%}'))
importance.head(12).rename('share of gain').to_frame().style.format('{:.1%}')
```

	share of gain
knn	96.6%
env	2.6%
depth	0.7%
patch	0.1%

Out[17]:

	share of gain
band_k16_gt10	60.0%
band_k16_median	16.2%
band_k64_gt10	6.5%
band_k4_median	3.1%
band_k256_gt10	2.0%
any_k16_median	1.8%
band_k4_gt10	1.4%
band_k4_distance	1.1%
band_k64_distance	0.8%
band_k16_distance	0.7%
band_k256_distance	0.7%
depth	0.7%

Summary

The pipeline. The Python loader reproduces the Node loader byte for byte (section 1), and so does the name correction tool. Adding a data set is now: copy the CSV into `data/sources/`, write its preprocessor, find its name corrections, and run `prep.py build`.

ARRP in production does nothing. Production's `training-data.json` has 7.4M wells, but none of ARRP's 6.5M match a map mouza (ARRP's union and mouza names are in capitals), so model6 never uses them: the production model is, file for file, the pre-ARRP model (A = B). Locating ARRP by GPS, with depths in metres, makes 99.8% of the wells usable (C).

Before and after (dissertation scoring, polluted > 10 ppb; F1, with sensitivity in brackets):

test set	A: production	C: with ARRP	C1: one test per well	C2: ARRP balanced
all held-out wells (1.49M, 88% ARRP)	0.648 (0.78)	**0.776** (0.77)	0.774 (0.77)	0.767 (0.77)
held-out ARRP wells (1.31M, field kits, 2021-23)	0.625 (0.77)	**0.776** (0.77)	0.774 (0.77)	0.764 (0.77)
held-out older wells (1Mdataset, BGS, Danida)	**0.810** (0.81)	0.774 (0.74)	0.778 (0.75)	0.788 (0.77)
BGD Traverse (213 lab-tested wells)	**0.685** (0.57)	0.545 (0.38)	0.555 (0.39)	0.570 (0.41)
VGQD (173 lab-tested wells)	**0.658** (0.54)	0.520 (0.37)	0.459 (0.31)	0.459 (0.31)

model6 A reproduces the dissertation's model5 numbers on the older wells (F1 0.81 vs 0.82), so the evaluation matches the earlier work.

With ARRP, model6 is much better on today's wells overall (accuracy 0.70 → 0.84), mostly by giving far fewer false alarms, but it predicts *safer*: it misses more polluted wells among the older wells and in both lab-tested sets (on BGD Traverse it catches 42 instead of 63 of the 110 polluted wells). For a screening tool that is the part that matters, so **C should not replace the production model as it is.**

Pruning ARRP doesn't fix it. Keeping one test per well (C1) changes almost nothing; balancing ARRP against the older data (C2) recovers a little on the older wells but not on the lab-tested sets. The reason is in ARRP's readings, not its volume: in the same unions and depth bands as the lab-tested wells, the older field-kit surveys agree with the lab (about 50% over 10 ppb) and ARRP reads about 20 points lower (about 31%). So either ARRP's field kits under-read, or today's wells really differ from the lab-tested ones. Before ARRP goes into production, its readings need checking against lab results, for example by lab-testing a sample of ARRP wells, or by calibrating ARRP's readings against the places where lab data exists.

The two lab-tested sets are small (a few dozen polluted red-stained wells each), but every comparison points the same way.

What else the data could give (section 7). Gradient boosting on the nearest wells' arsenic (by GPS, in the same depth band) reaches F1 0.818 on the ARRP wells and 0.819 on the older wells (polluted > 10 ppb), against model6 C's 0.776 and 0.773 on the same wells; even "the median of the 16 nearest wells" gets 0.80. Almost all the gain is from local data with real coordinates. model6's expert patch on its own gets 0.64 / 0.61, and adds nothing once the nearest wells are known; the environmental rasters alone get 0.75 / 0.77. So:

- there is clear room to do better than model6's mouza-level medians, mostly from finer location;
- a CNN looking at the wells around each place would need to beat this gradient boosting baseline, not model6;
- the next tests should be (1) a spatial hold-out (whole upazilas or 20 km blocks), because a random hold-out flatters methods that use nearby wells, and (2) lab-tested wells, given the field-kit question above. The quick-look models were only scored on held-out wells, since the independent sets are too small to tell them apart.